Technical Architecture & Implementation Document

System Name: Directorate of Environment and Climate Change (DECC) Application Intelligence & Environmental Review Portal
Document Version: 2.0.0
Classification: Technical Architecture & Implementation Guide
Status: Implemented & Production-Ready
Date: August 2026

Table of Contents

---

1. Executive Summary

The **Directorate of Environment and Climate Change (DECC) Review Portal** is an AI-powered enterprise review and decision-support platform built to automate the ingestion, verification, validation, and risk evaluation of environmental grant applications and statutory clearance requests.

Historically, environmental application processing relied on manual officer inspection of heterogeneous physical documents (PDF proposals, budget sheets, organizational certificates, and EIA reports), resulting in multi-week evaluation cycles, undetected budgetary contradictions, and compliance backlogs.

This platform bridges automated intelligence with strict statutory compliance through a **Human-in-the-Loop (HITL)** architecture:

- **Deterministic Validation & Cross-Document Consistency**: Eliminates data conflicts across submitted files.
- **Retrieval-Augmented Generation (RAG)**: Indexes statutory environmental scheme guidelines via semantic vector embeddings.
- **Machine Learning Ensemble**: Uses 3 production XGBoost models for multi-class risk classification, risk scoring (0–100), and application quality scoring (0–100).
- **Post-Scoring LLM Reasoning**: Generates structured, evidence-grounded case explanations without ever overriding deterministic scores.
- **Authoritative Officer Decision Cockpit**: Ensures that AI assists and informs, while the final statutory sign-off rests exclusively with authorized government officers.

---

2. System Objectives

1. **Deterministic Accuracy**: Ensure zero hallucinations in statutory compliance checks by computing rule outcomes deterministically.
2. **Cross-Document Integrity**: Detect budget discrepancies, timeline mismatches, and entity conflicts across multi-file submissions.
3. **Evidence-Grounded RAG**: Ground all policy guidelines and threshold limits directly in official government scheme documentation.
4. **Transparent Risk Scoring**: Provide calibrated risk indices (0–100) and feature importances to provide reviewers with explainable ML predictions.
5. **Operational Efficiency**: Reduce application turnaround time from weeks to minutes while maintaining full auditability.
6. **Bilingual Accessibility**: Full English and Hindi (`hi`) multilingual support across all officer workflows and reviewer interfaces.

---

3. Functional Scope

FUNCTIONAL AREA	IMPLEMENTED FUNCTIONALITY	PLANNED / FUTURE SCOPE
Application Intake	3-step submission wizard, metadata capture, multi-file upload (PDF, DOCX, XLSX, images).	Direct API integration with external citizen identity portals (DigiLocker, e-Pramaan).
Document Processing	Tesseract OCR, PyPDF/PDFPlumber text extraction, heuristic & LLM document classification.	Multi-page layout parsing with vision transformers for complex GIS maps.
Normalization	Unified `ApplicationProfile` synthesis, cross-document canonical field selection, conflict tracking.	Temporal entity resolution across multi-year historical applications.
Validation	12+ deterministic checks, scheme rule engine, cross-document comparison, RAG guideline checks.	Automated geospatial boundary overlap checks with GIS satellite layers.
RAG Knowledge Base	ChromaDB vector store, `sentence-transformers` embeddings, overlapping chunking, similarity retrieval.	Multi-vector parent-child document chunking for complex policy appendices.
Machine Learning	3-model XGBoost ensemble (`risk_classifier` , `risk_regressor` , `quality_regressor`), 13 canonical features.	Continual active learning loops triggered by officer decision overrides.
LLM Reasoning	OpenRouter OpenAI-compatible client, structured Pydantic schema validation, fallback retry.	On-premise air-gapped LLM inference via vLLM / Ollama for classified files.
Reviewer Cockpit	Priority queue, evidence inspection, decision recording (Approve / Clarify / Reject), notes.	Multi-tier sequential approval hierarchies (Inspector → Director → Secretary).
Notifications & Reports	SMTP email dispatch of rich HTML gap-free clearance reports with actionable issue cards.	SMS gateway integration and WhatsApp status notification webhooks.

4. Non-Functional Requirements

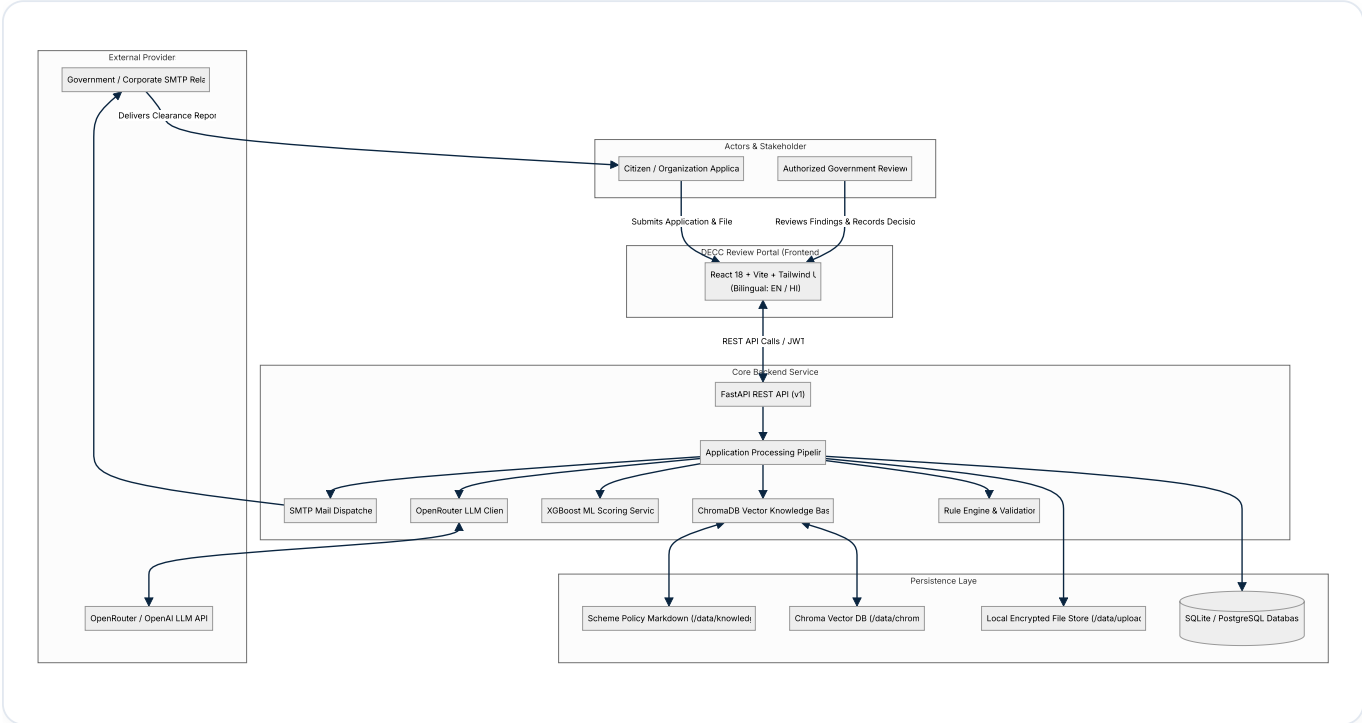
- **Availability & Resilience**: The processing pipeline must be fault-tolerant; if the external LLM provider experiences timeouts or rate limits (HTTP 429), the pipeline flags degraded mode, preserves all deterministic checks, and halts gracefully without data loss.
- **Security & Privacy**: Zero storage of raw LLM API keys in logs or client-facing responses; role-based access control (RBAC); strict MIME-type and checksum file validation.
- **Performance**: End-to-end processing of a 4-document application package within 15–45 seconds depending on OCR and LLM response latency.

- **Auditability**: Complete append-only audit trail logging every state transition, user interaction, and ML prediction with ISO timestamps.

---

5. System Context

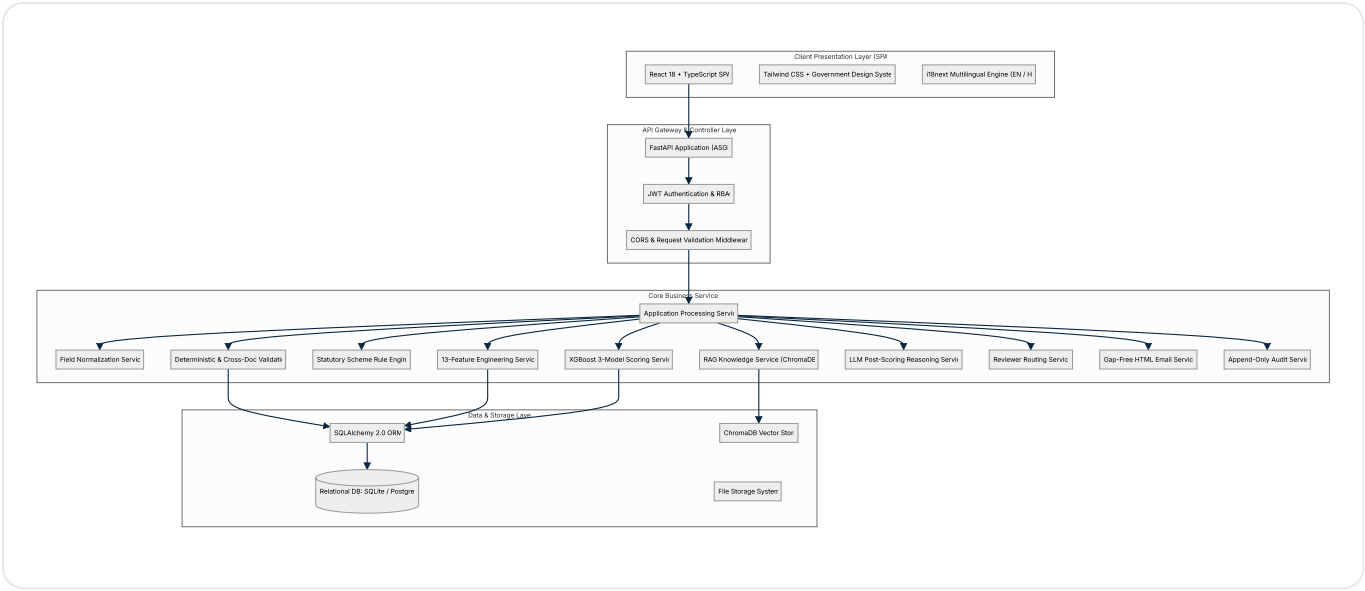
The system interfaces with applicants, government review officers, external LLM inference providers, SMTP mail relays, and internal vector/relational databases.



---

6. High-Level Architecture

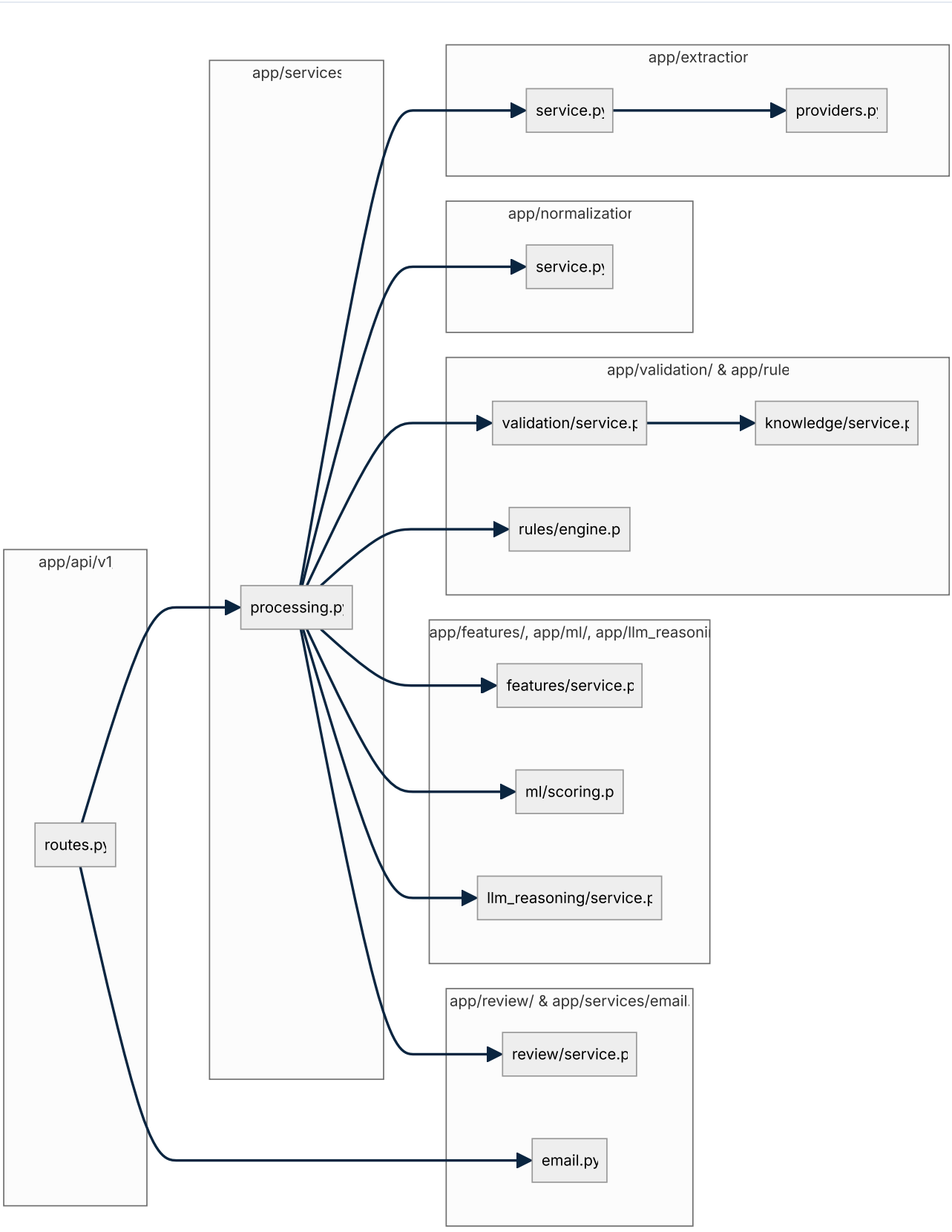
The platform follows a layered, modular service-oriented architecture (SOA) with a clean separation between ingestion, extraction, validation, machine learning, generative reasoning, and user presentation.



7. Detailed Component Architecture

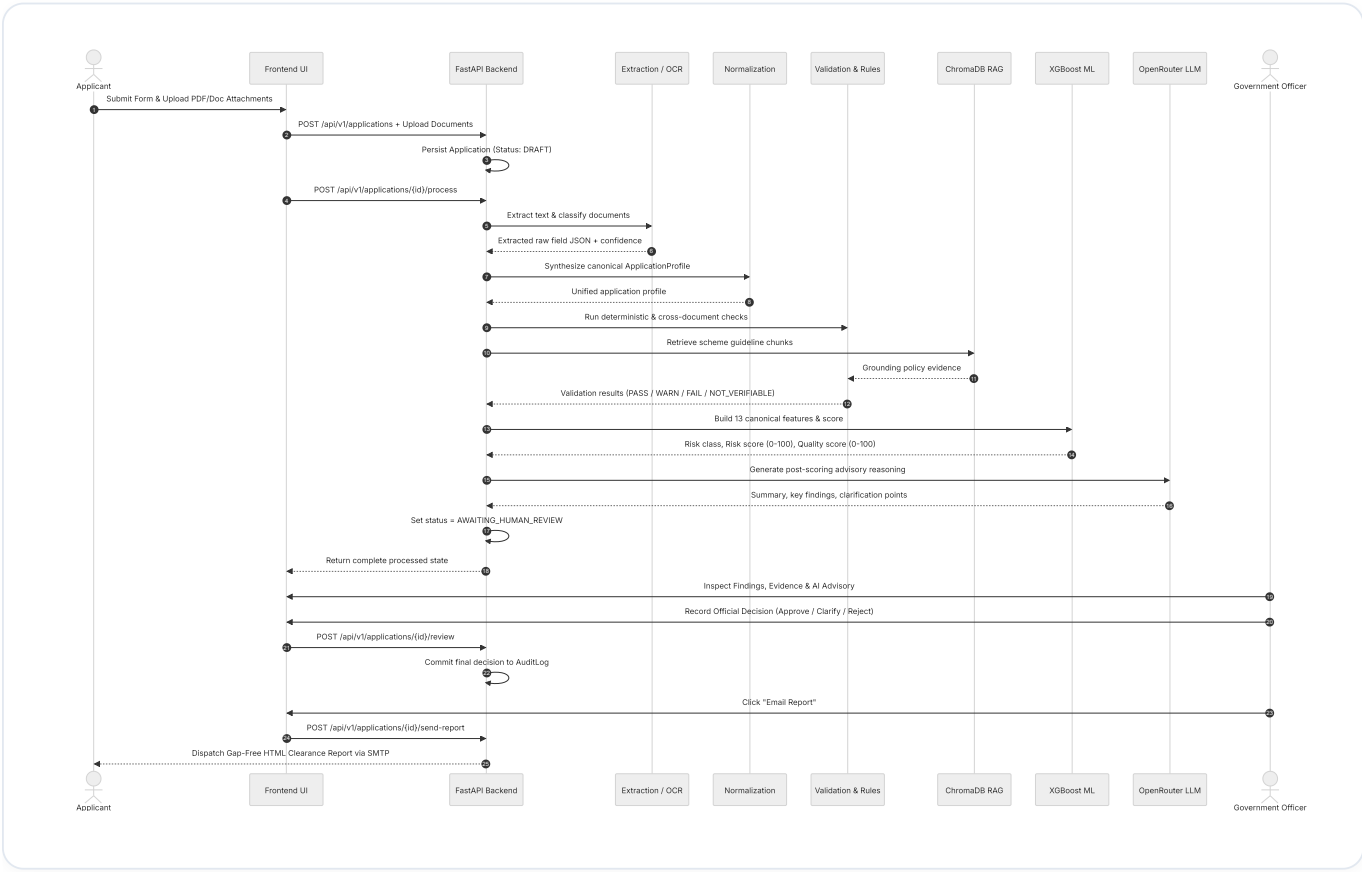
Below is the directory-level mapping of backend components and their functional responsibilities:

```
backend/app/
├── api/v1/routes.py           # REST endpoints: Auth, Applications, Documents, Processing, Scoring, R
├── core/
│   ├── config.py              # Pydantic BaseSettings (OpenRouter, SMTP, DB paths, OCR, thresholds)
│   ├── exceptions.py          # Domain exceptions (ModelUnavailableError, LLMProviderError, OCRProvid
│   └── security.py            # JWT issuance, password hashing, UserContext dependency
├── db/
│   ├── base.py                # SQLAlchemy declarative base
│   └── session.py             # Engine factory, session maker, SQLite auto-migration helper
├── models/entities.py         # 18 SQLAlchemy ORM entity models
├── extraction/
│   ├── service.py             # DocumentIntelligenceService coordinator
│   └── providers.py           # PDFPlumber, Tesseract OCR, Heuristic Regex, LLM extraction providers
├── normalization/service.py   # Multi-document field merging, canonical selection, ApplicationProfile
├── validation/service.py      # Deterministic checks, RAG guideline checks, cross-document comparison
├── rules/engine.py            # SchemeRule evaluation against state guidelines and financial limits
├── knowledge/
│   └── service.py             # ChromaKnowledgeBase, sentence-transformers embeddings, Markdown chunk
├── features/service.py        # 38 pipeline features → 13 canonical ML features for XGBoost
├── ml/
│   ├── scoring.py             # XGBoostScoringService (3 .ubj models: classifier, risk_reg, quality_r
│   └── application_intelligence_xgboost_training_artifacts/ # Serialized UBJ models & schema JSON
├── llm_reasoning/service.py   # Post-scoring OpenRouter client, Pydantic validation, exponential back
├── review/service.py          # ReviewService: decision recording, override logging, audit updates
├── routing/service.py         # Auto-assigns applications to reviewer roles based on risk and confide
├── services/
│   ├── processing.py          # Master ApplicationProcessingService pipeline coordinator
│   ├── email.py               # SmtpEmailService: gap-free HTML clearance report generation & dispatc
│   └── seed.py                # Seeds default environmental schemes and statutory guidelines
└── workflow/
    ├── graph.py               # LangGraph state machine definition (12 nodes)
    └── state.py               # ApplicationProcessingState TypedDict
```

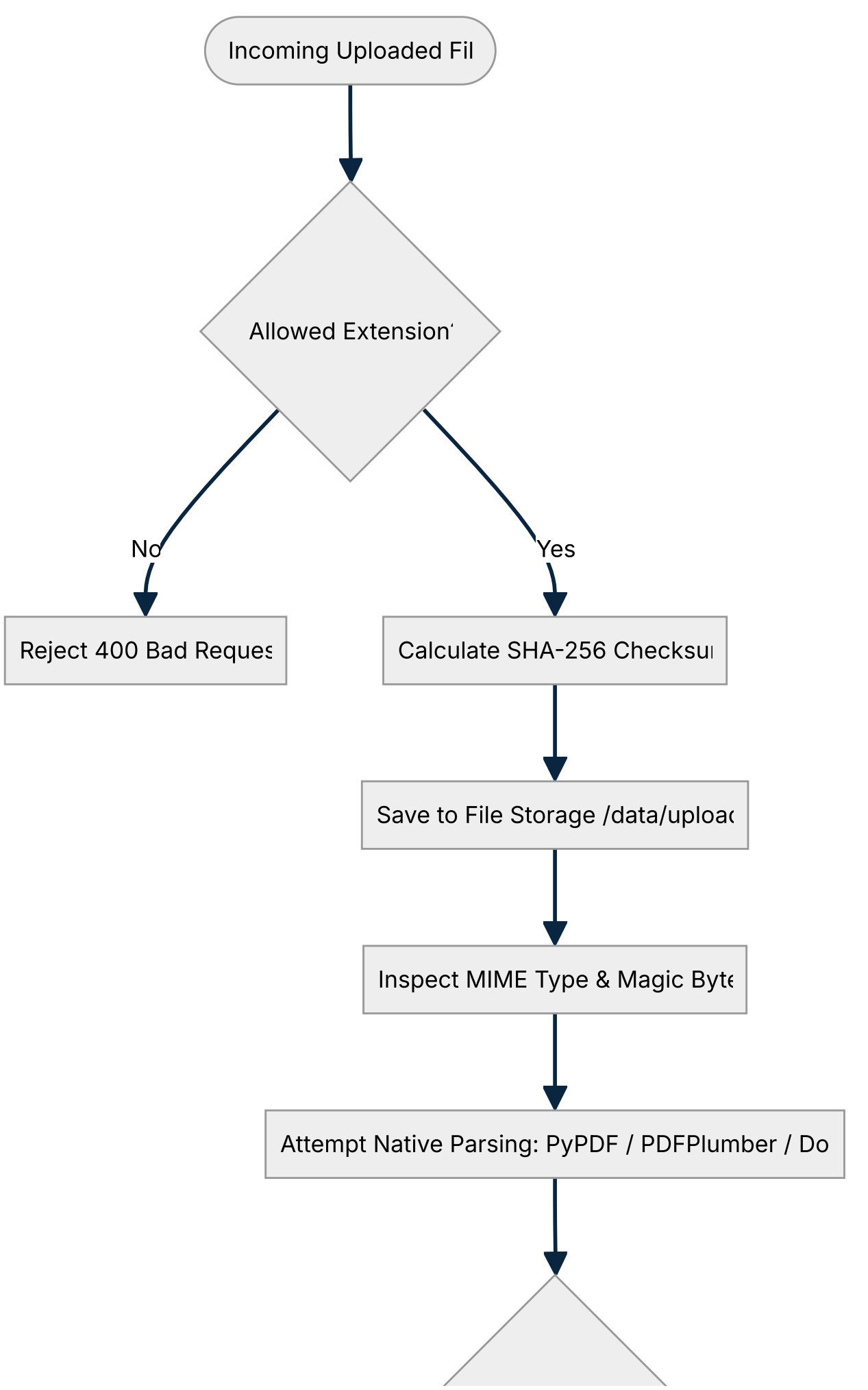

app/services
app/extractior
service.py
providers.p
app/normalizatior
service.py
app/validation/ & app/rule
validation/service.p
knowledge/service.p
rules/engine.p
app/api/v1
processing.p
app/features/, app/ml/, app/llm_reasoni
features/service.p
ml/scoring.p
routes.py
llm_reasoning/service.p
app/review/ & app/services/email
review/service.p
email.py

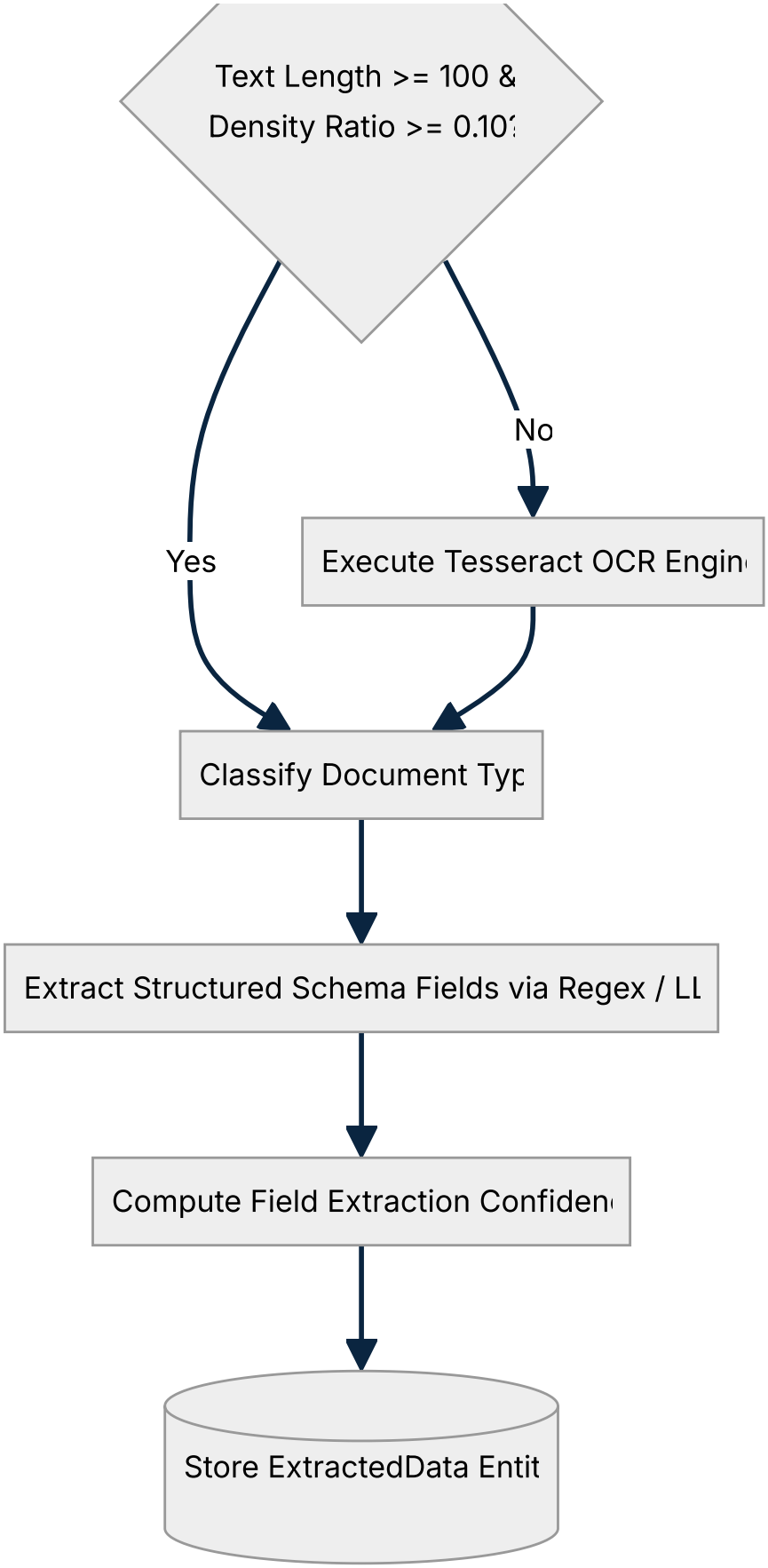
8. End-to-End Application Workflow

The lifecycle of an application progresses through 7 human stages (orchestrated internally across 12 LangGraph nodes).



9. Document Processing Workflow


Incoming Uploaded Fil
Allowed Extension
No
Yes
Reject 400 Bad Reques
Calculate SHA-256 Checksu
Save to File Storage /data/uploa
Inspect MIME Type & Magic Byte
Attempt Native Parsing: PyPDF / PDFPlumber / Do


Text Length >= 100 &
Density Ratio >= 0.10
No
Yes
Execute Tesseract OCR Engin
Classify Document Typ
Extract Structured Schema Fields via Regex / LL
Compute Field Extraction Confiden
Store ExtractedData Entit

10. OCR & Extraction Architecture

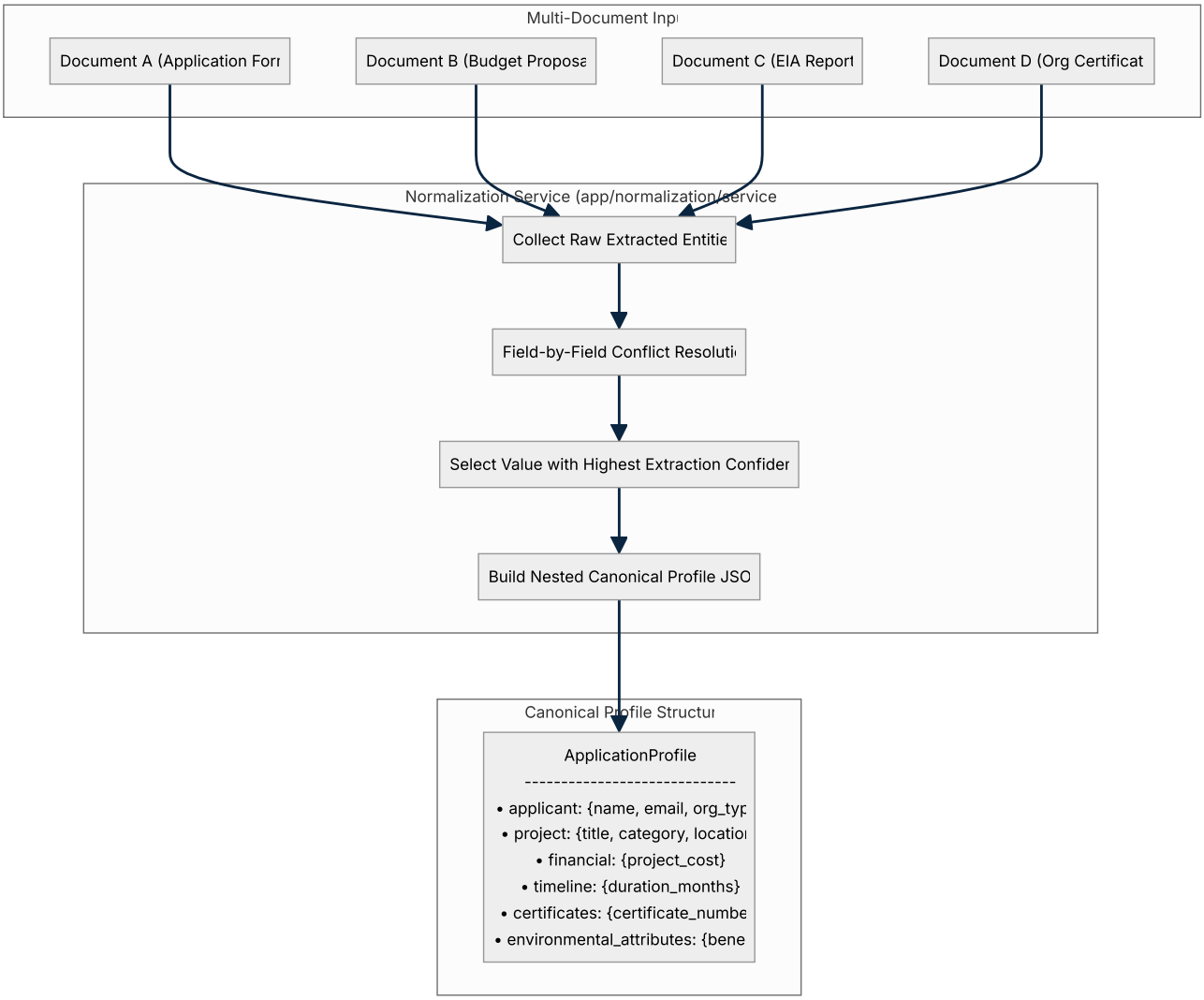
The extraction subsystem employs a tiered strategy:

1. **Direct Digital Extraction**: `pdfplumber` and `pypdf` extract native text layers, bounding boxes, and table cells.
2. **Quality Metric Verification**: Computes `text_length` and `chars_to_bytes_ratio`. If `text_length < 100` or `ratio < 0.10`, the file is marked as a scanned raster image.
3. **Tesseract OCR Fallback**: Renders PDF pages to 300 DPI PIL images and executes `pytesseract.image_to_string` with language pack `eng`.
4. **Structured Key Extraction**:
    - `extract_applicant_name`, `extract_project_title`, `extract_cost`, `extract_duration`, `extract_certificate_number`.
    - Heuristic regex patterns execute first. If `LLM_PROVIDER` is active, high-confidence LLM parsing extracts nested entities.

---

11. Normalization & Data Processing

When multiple documents are uploaded (e.g. Application Form, Project Proposal, Budget Sheet, Certificate), extraction produces separate `ExtractedData` records with potentially conflicting values.



Conflict Resolution Strategy:

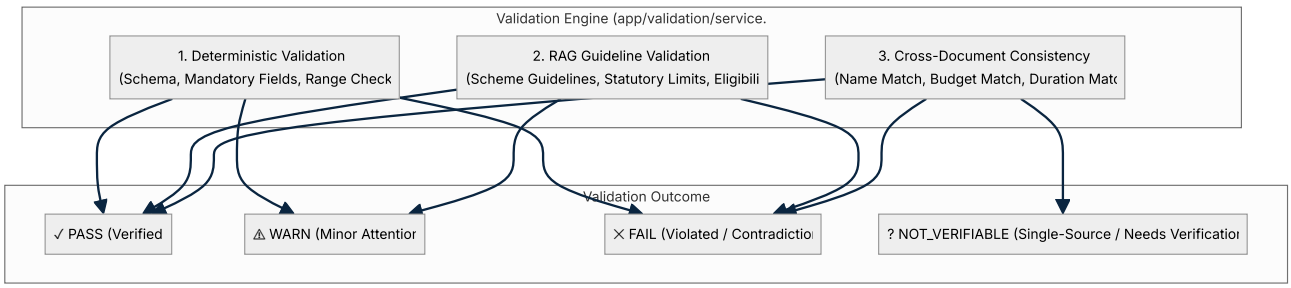
- For scalar strings (applicant name, project title), the value with the highest confidence extraction is selected as canonical, while all alternate values are stored in `sources` arrays for cross-document consistency checks.
- For financial values (costs), numbers are parsed into floating-point INR amounts. Discrepancies between documents exceeding ₹5,000 or 2% trigger a `CROSS_DOCUMENT_CONSISTENCY` warning.

---

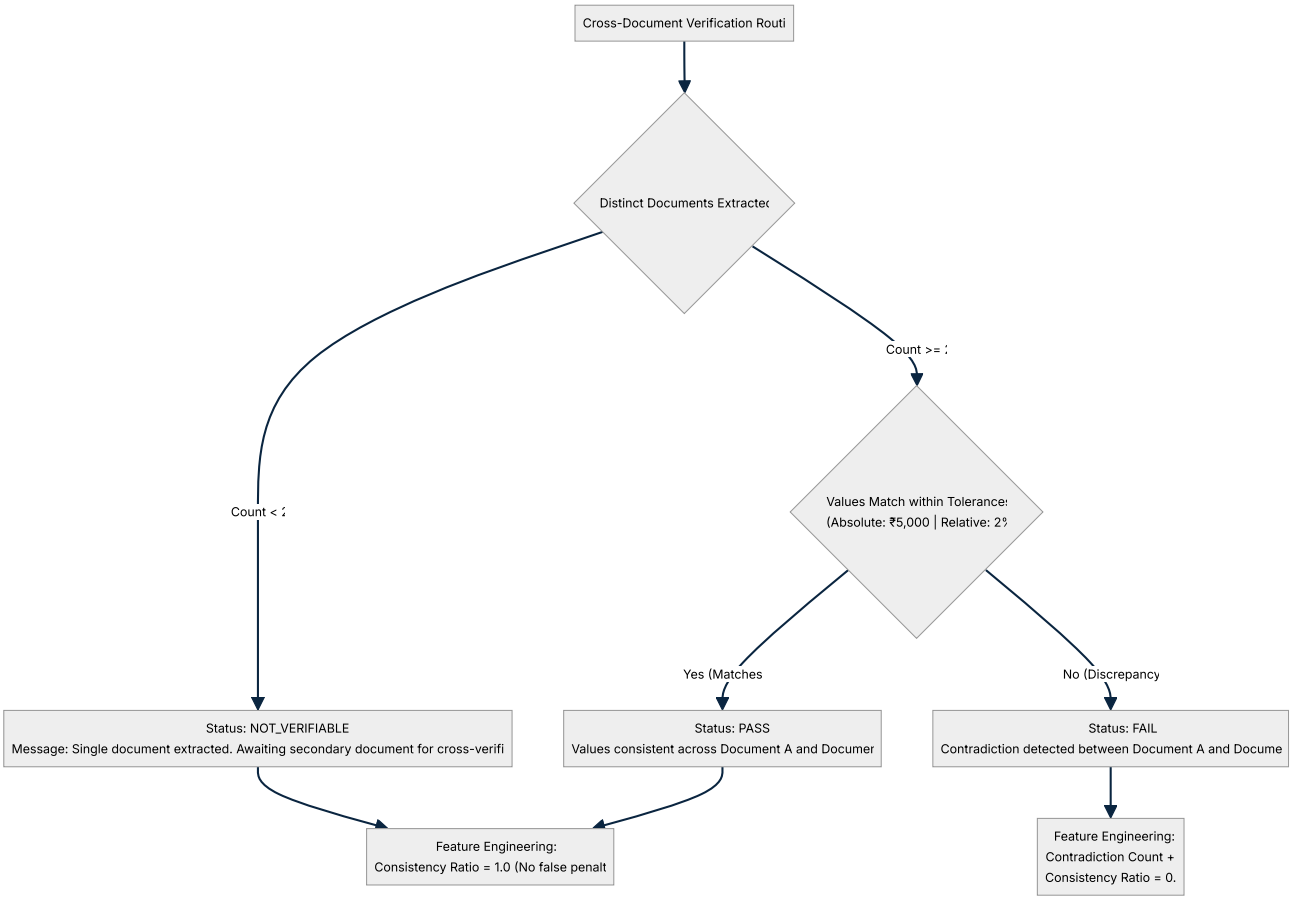
12. Validation Architecture

Validation executes across **three distinct layers**, producing deterministic findings categorized into 4 statuses:

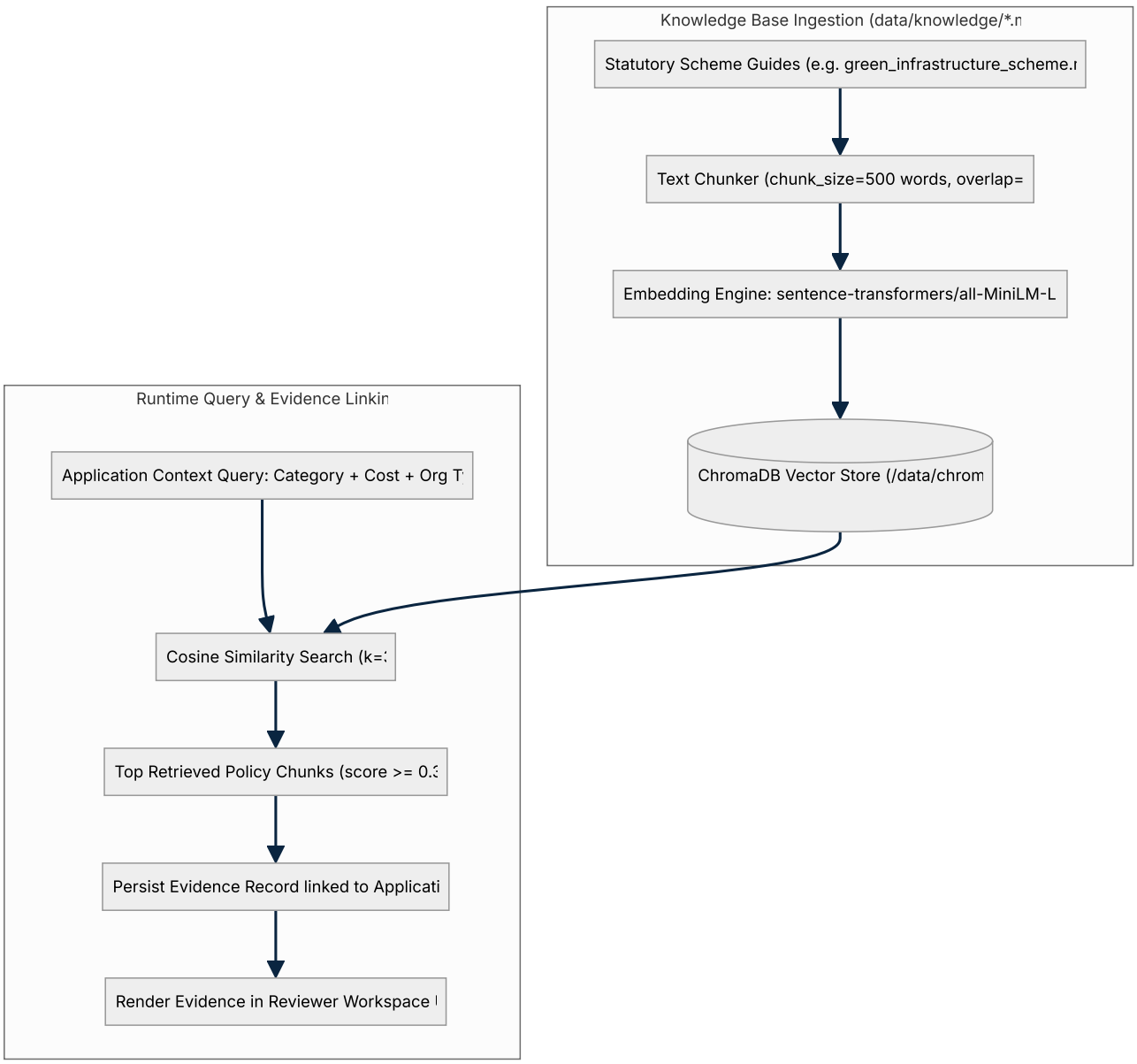
- `PASS`: Condition strictly met with positive evidence.
- `WARN`: Minor discrepancy or missing non-critical document.
- `FAIL`: Explicit violation of scheme thresholds or cross-document contradiction.
- `NOT_VERIFIABLE`: Required secondary document missing for multi-document cross-comparison. **Crucially,** `NOT_VERIFIABLE` **is never penalized as a contradiction.**



13. Cross-Document Verification



14. RAG & Knowledge Base Architecture



15. ML Feature Engineering & XGBoost Scoring

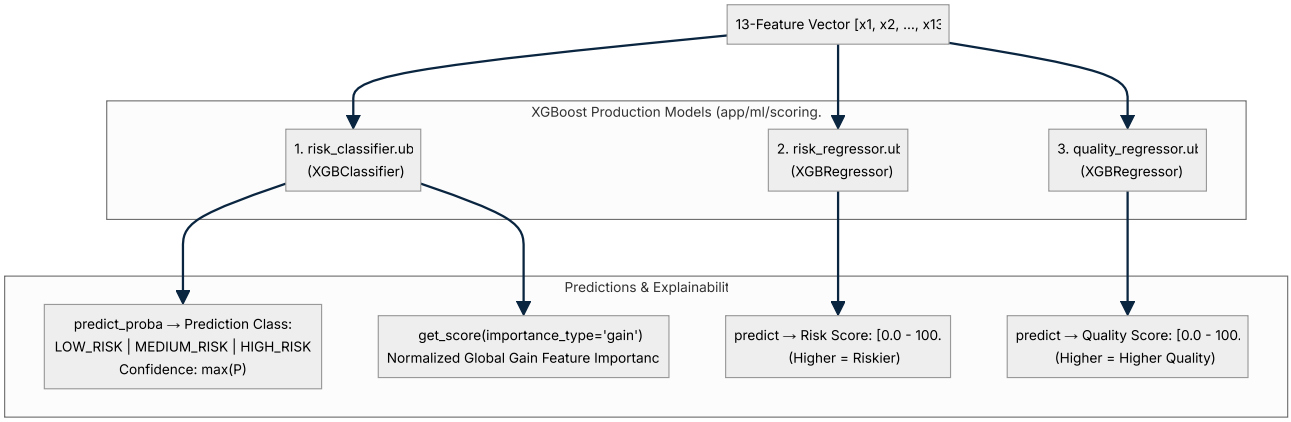
The platform transforms raw validation results, scheme rules, and application profile parameters into **38 available features**, isolating **13 canonical features** required by the production XGBoost models.

The 13 Canonical ML Features

#	FEATURE NAME	SOURCE	TRANSFORMATION	RANGE	MEANING & MISSING HANDLING
1	`document_completeness`	Document Check	`(required - missing) / required`	`[0.0, 1.0]`	Ratio of mandatory documents attached. Defaults to 0.0 if no documents.
2	`required_field_completeness`	Field Validation	`passed_fields / total_required_fields`	`[0.0, 1.0]`	Completeness of mandatory application form fields.
3	`eligibility_pass_ratio`	Scheme Rules	`passed_rules / total_rules`	`[0.0, 1.0]`	Proportion of scheme rules satisfied.
4	`budget_consistency`	Cross-Doc Validation	Cost consistency check ratio	`[0.0, 1.0]`	1.0 if costs match or single source; 0.0 if contradicted.
5	`certificate_validity`	Authenticity Check	Binary flag	`{0.0, 1.0}`	1.0 if certificate number verified; 0.0 if invalid or failed.
6	`contradiction_count`	Cross-Doc Validation	Integer count cast to float	`[0.0, N]`	Number of explicit cross-document data conflicts detected.
7	`duplicate_similarity`	Checksum Check	Binary flag	`{0.0, 1.0}`	1.0 if duplicate document checksum exists in database.
8	`suspicious_indicator_count`	Anomaly Check	Count of indicators	`[0.0, N]`	Count of anomaly flags (e.g. excessive

#	FEATURE NAME	SOURCE	TRANSFORMATION	RANGE	MEANING & MISSING HANDLING
					cost > ₹1 crore).
9	`document_quality`	OCR & Validation	`(pass + not_checked*0.25) / total`	`[0.0, 1.0]`	Overall document legibility and check pass rate.
10	`proposal_quality`	Composite	`(field_comp + extract_conf + val_conf) / 3`	`[0.0, 1.0]`	Synthesized metric of application detail and extraction fidelity.
11	`project_feasibility`	Composite	`(rule_pass + budget_cons + dur_cons) / 3`	`[0.0, 1.0]`	Feasibility of scope matching statutory scheme limits.
12	`environmental_impact`	Normalization	Binary flag	`{0.0, 1.0}`	1.0 if environmental benefits and mitigation scope defined.
13	`extraction_confidence`	Normalization	OCR / Parser average score	`[0.0, 1.0]`	Mean extraction confidence across all parsed fields.

3-Model XGBoost Ensemble Pipeline

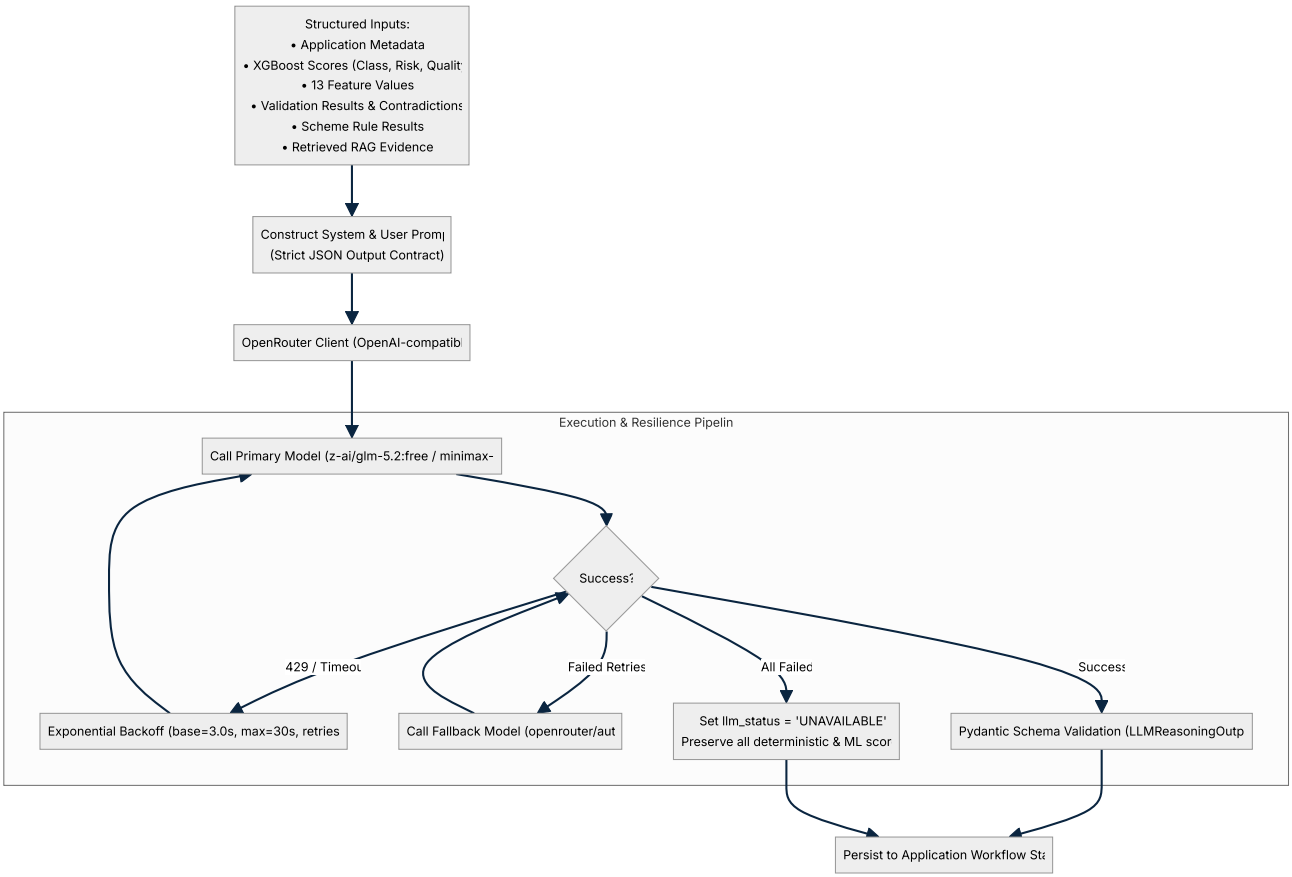


> [!NOTE] Feature importance is calculated using **global gain importance** from the XGBoost booster trees, normalized by total gain. It does not perform per-application SHAP attribution.

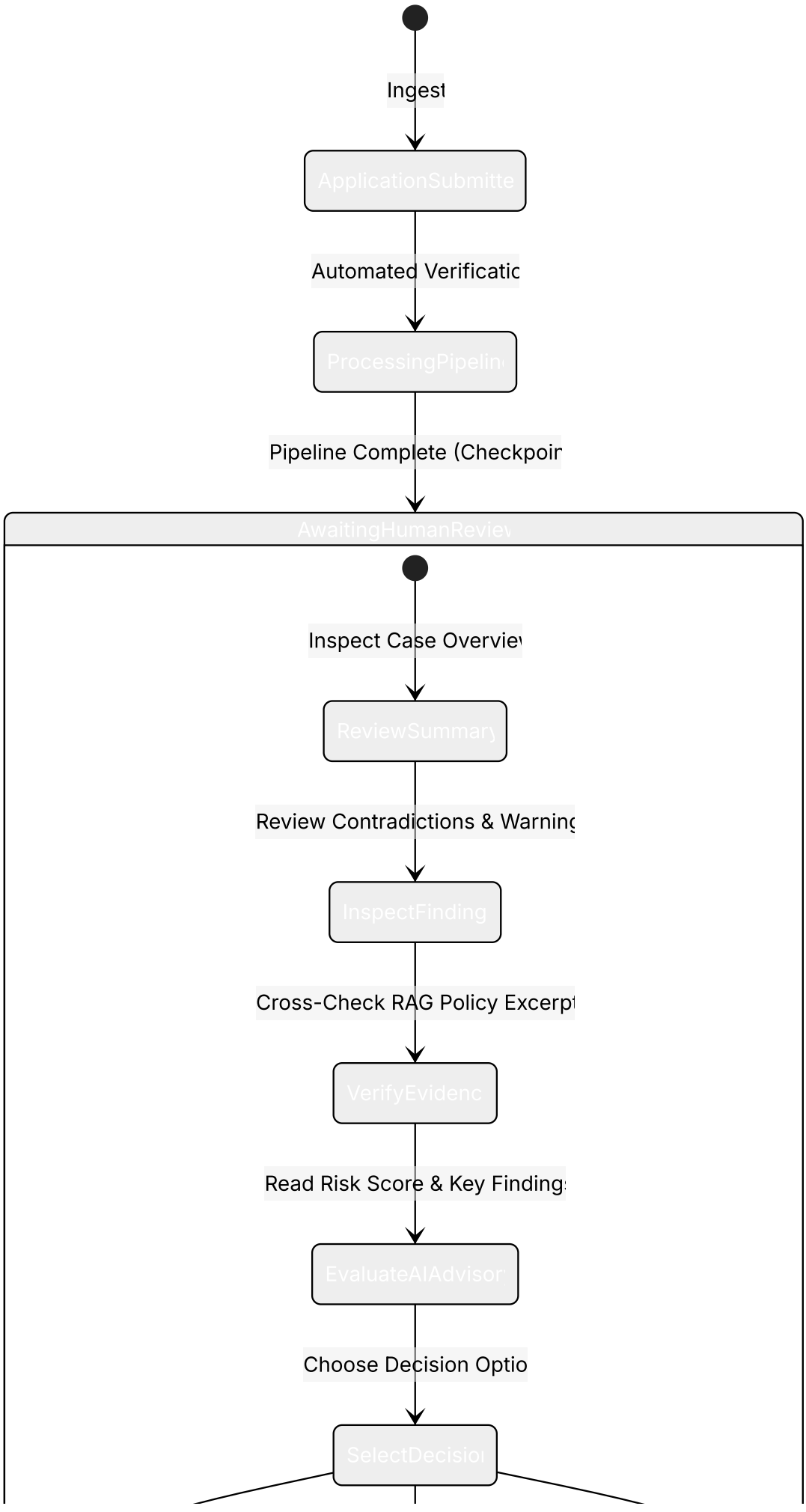
---

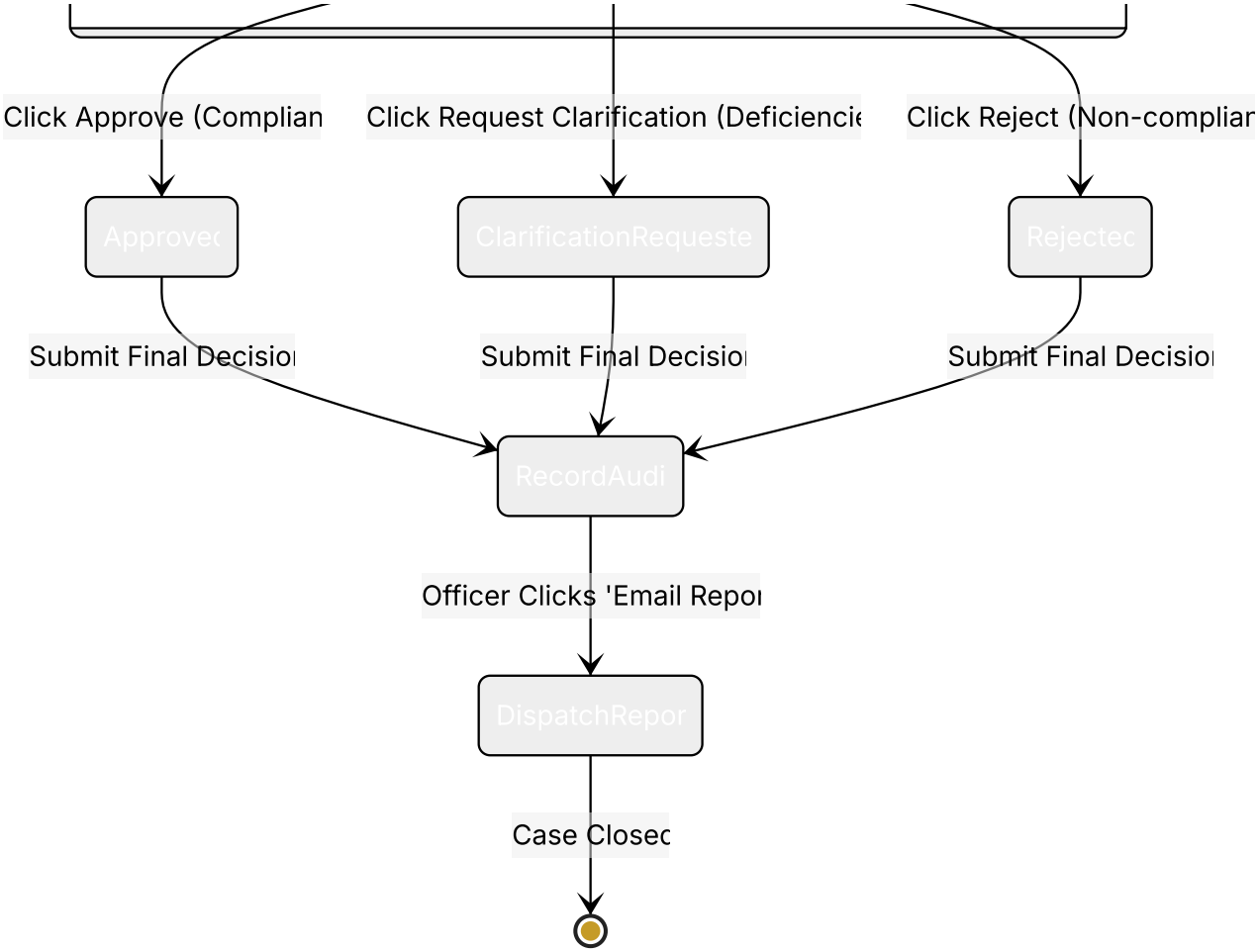
16. LLM Reasoning Architecture

The LLM is invoked **only once** after deterministic validation and XGBoost scoring have concluded. It provides post-scoring decision support and never alters scores.


Structured Inputs:
• Application Metadata
• XGBoost Scores (Class, Risk, Qualit
• 13 Feature Values
• Validation Results & Contradictions
• Scheme Rule Results
• Retrieved RAG Evidence
Construct System & User Prom
(Strict JSON Output Contract)
OpenRouter Client (OpenAI-compatib
Execution & Resilience Pipelin
Call Primary Model (z-ai/glm-5.2:free / minimax-
Success?
429 / Timeou
Failed Retries
All Failed
Success
Exponential Backoff (base=3.0s, max=30s, retries
Call Fallback Model (openrouter/aut
Set llm_status = 'UNAVAILABLE'
Preserve all deterministic & ML scor
Pydantic Schema Validation (LLMReasoningOutp
Persist to Application Workflow Sta

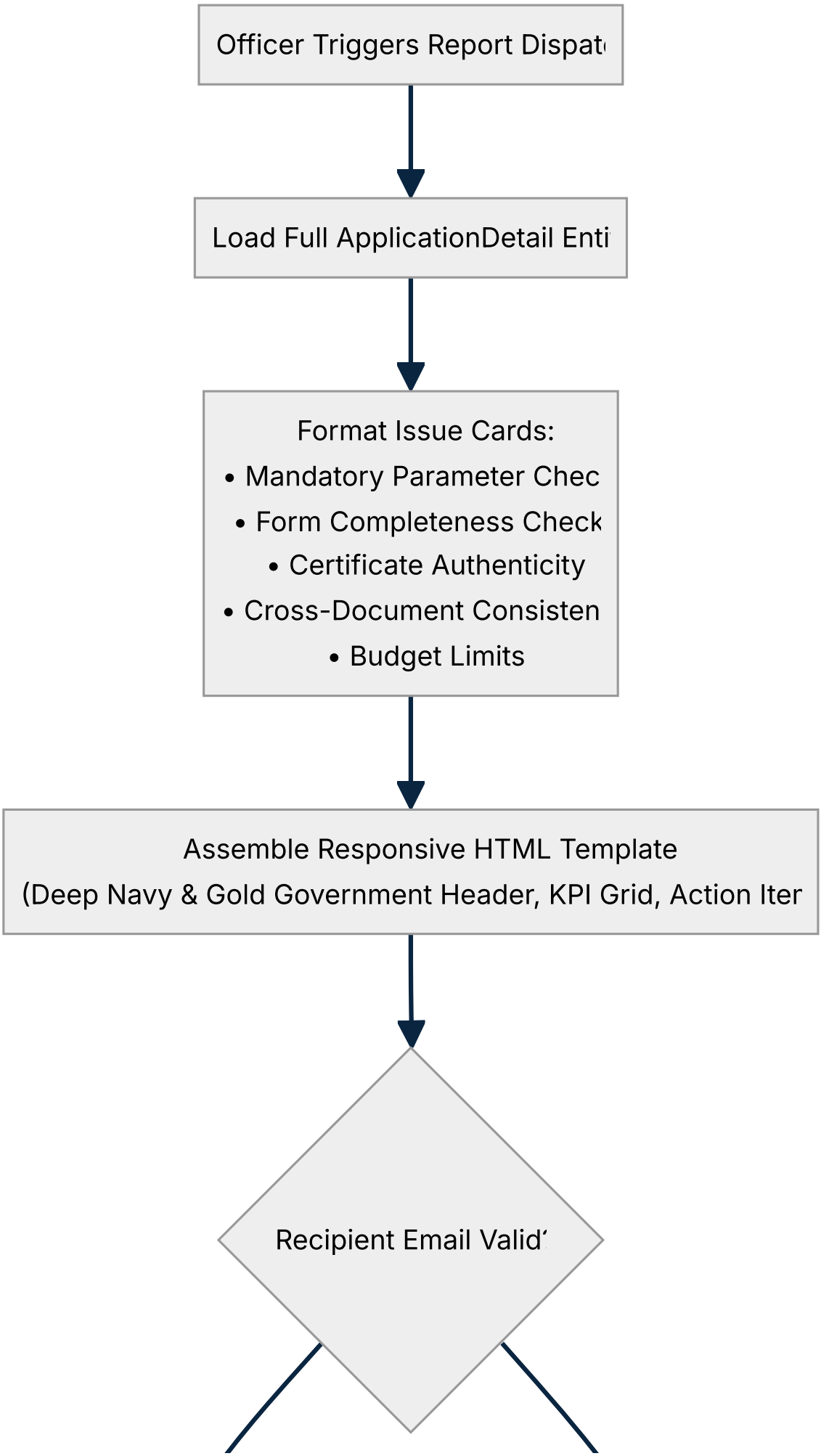
17. Human Reviewer Workflow

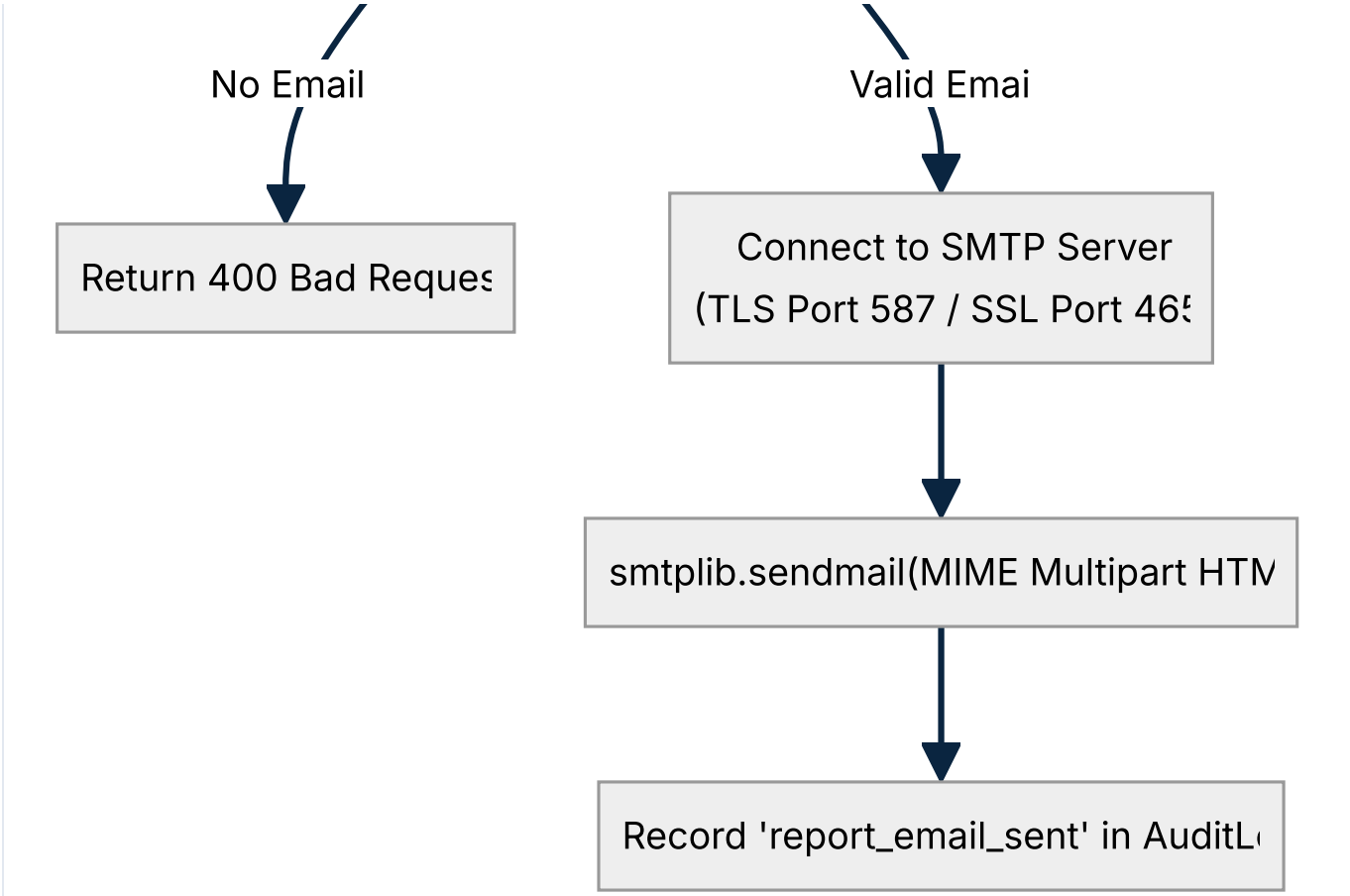

Ingest
ApplicationSubmitte
Automated Verificatic
ProcessingPipelin
Pipeline Complete (Checkpoir
AwaitingHumanReviev
Inspect Case Overvie
ReviewSummary
Review Contradictions & Warning
InspectFinding
Cross-Check RAG Policy Excerpt
VerifyEvidenc
Read Risk Score & Key Findings
EvaluateAIAdvisor
Choose Decision Optio
SelectDecisio



18. Email & Conclusion Workflow

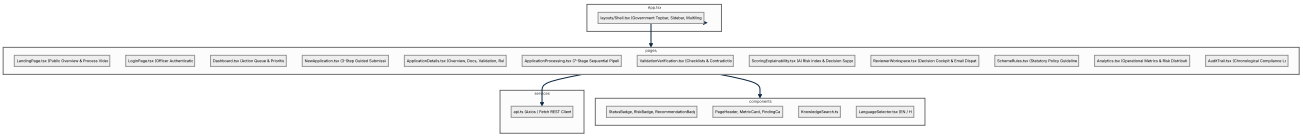
When an officer records a decision or clicks **"Email Report"**, the system generates a gap-free, responsive HTML email report delivered via SMTP.


Officer Triggers Report Dispat
Load Full ApplicationDetail Enti
Format Issue Cards:
• Mandatory Parameter Chec
• Form Completeness Check
• Certificate Authenticity
• Cross-Document Consisten
• Budget Limits
Assemble Responsive HTML Template
(Deep Navy & Gold Government Header, KPI Grid, Action Iter
Recipient Email Valid



19. Frontend Architecture

The frontend is a single-page application built on **React 18**, **TypeScript**, and **Tailwind CSS**, strictly enforcing the Government Environmental Review Portal design system.



20. Backend Architecture

The backend is built with **FastAPI** and **Python 3.11+**, structured with strict dependency injection, domain services, and repository layers.

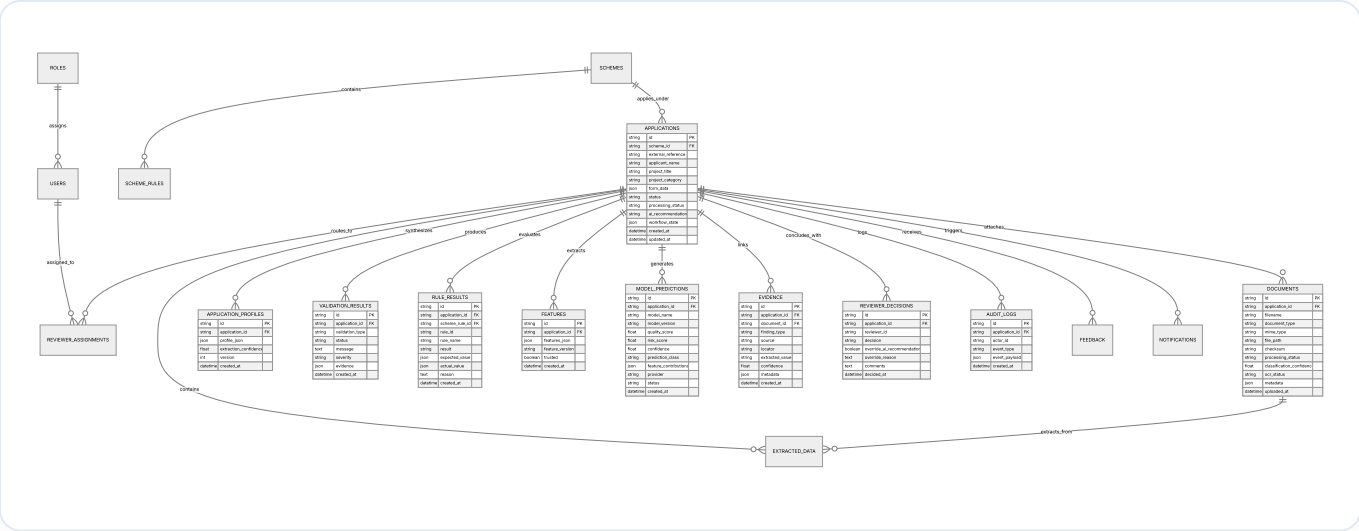
- **FastAPI ASGI Server**: High-throughput async request handling.
- **SQLAlchemy 2.0 ORM**: Type-annotated mapped columns, explicit relationships, and SQLite/Postgres compatibility.

- **Pydantic v2**: Strict payload validation, serialization, and schema enforcement.
- **Modular Services**: Independent service singletons (`validation_service` , `feature_engineering_service` , `prediction_service` , `llm_reasoning_service`).

---

21. Database Architecture & Data Model

Below is the complete entity-relationship diagram representing all 18 database entities:



---

22. API Architecture & Endpoints

CATEGORY	HTTP METHOD	ENDPOINT PATH	DESCRIPTION
Auth	`POST`	`/api/v1/auth/token`	Issue demo / testing JWT access token.
Auth	`GET`	`/api/v1/auth/me`	Fetch authenticated user role and identity.
Applications	`POST`	`/api/v1/applications`	Create a new environmental clearance application.
Applications	`GET`	`/api/v1/applications`	List all applications ordered by creation date.
Applications	`GET`	`/api/v1/applications/{id}`	Get complete application detail with predictions and findings.
Applications	`DELETE`	`/api/v1/applications/{id}`	Permanently delete application and cascading records.
Documents	`POST`	`/api/v1/applications/{id}/documents`	Upload a PDF, DOCX, XLSX, or image attachment.
Documents	`DELETE`	`/api/v1/documents/{id}`	Remove an uploaded document from an application.
Processing	`POST`	`/api/v1/applications/{id}/process`	Execute full automated processing pipeline.
Validation	`GET`	`/api/v1/applications/{id}/validation`	Retrieve deterministic, RAG, and cross-doc check results.
Scoring	`GET`	`/api/v1/applications/{id}/score`	Fetch XGBoost predictions, risk scores, and gain contributions.
Evidence	`GET`	`/api/v1/applications/{id}/evidence`	List all extracted evidence citations and policy chunks.
Review	`POST`	`/api/v1/applications/{id}/review`	Submit official reviewer decision (Approve / Clarify / Reject).
Email	`POST`	`/api/v1/applications/{id}/send-report`	Dispatch clearance report to applicant email via SMTP.
Knowledge	`GET`	`/api/v1/knowledge/search`	Query scheme guideline chunks via vector similarity.
Analytics	`GET`	`/api/v1/analytics/overview`	Fetch system-wide metrics, risk distribution, and throughput.

23. Authentication & Authorization

- **JWT Bearer Authentication**: Signed using `HS256` with configurable `JWT_SECRET` and expiration (`JWT_EXPIRE_MINUTES`).
- **Role Hierarchy**:
  - `admin`: Full system configuration, scheme rule modifications.
  - `senior_reviewer`: High-risk application review authority, decision overrides.
  - `reviewer`: Standard case review, clarification dispatch.
  - `viewer`: Read-only access to audit logs and analytics.
- **Configurable Enforcement**: Controlled via `AUTH_ENABLED` in `backend/.env`. When disabled for local demonstrations (`DEMO_MODE=true`), default test identities are injected cleanly.

---

24. Error Handling & Resilience

The platform implements explicit exception hierarchies rather than generic try-catch blocks:

---

25. LLM Rate Limiting & Retry Architecture

- **Primary Model**: Configured via `LLM_MODEL` (e.g. `minimax/minimax-m3`, `z-ai/glm-5.2:free`).
- **Fallback Model**: Configured via `LLM_FALLBACK_MODEL` (e.g. `openrouter/auto`).
- **Retry Backoff Algorithm**: $$\text{Delay} = \min(\text{LLM_RETRY_BACKOFF_SECONDS} \times 2^{\text{attempt}}, 30.0)$$
- **Max Retries**: 2 retries per model before switching to fallback model.
- **Degraded Mode Protection**: If all models fail, `state["degraded_mode"] = True` is logged. Deterministic checks and XGBoost predictions remain intact.

---

26. Logging & Observability

Every pipeline execution produces standardized structured logs:

```
[PIPELINE] application=APP_ID stage=INGEST status=STARTED documents=4
[PIPELINE] application=APP_ID stage=EXTRACT document=DOC_ID provider=pdfplumber confidence=0.920 status
[FEATURE_ENGINEERING] features=13/13 total=38
[FEATURE_ENGINEERING] feature=document_completeness value=1.0000
[ML_SCORING] model_status=READY features=13/13 prediction_class=LOW_RISK risk_score=15.2 quality_score=
[PIPELINE] application=APP_ID stage=LLM_REASONING llm_status=COMPLETED duration_ms=1850
[PIPELINE] application=APP_ID status=AWAITING_HUMAN_REVIEW reviewer_role=senior_reviewer recommendation
```

27. Data Flow Tables

STAGE	INPUT DATA	TRANSFORMATION / PROCESSING	OUTPUT DATA	PERSISTED ENTITY
INGEST	Uploaded Multipart File	Checksum calculation, MIME validation, storage	File path on disk	`Document`
EXTRACT	Stored File on Disk	Text parsing, OCR, entity regex, classification	Raw field key-values	`ExtractedData`
NORMALIZE	Multiple `ExtractedData`	Canonical merging, conflict resolution	Unified JSON Profile	`ApplicationProfile`
VALIDATE	Profile + Scheme Rules	Schema validation, cross-doc comparison, RAG	Validation checklist	`ValidationResult`, `Evidence`
RULE_EVAL	Profile + Rules DB	Deterministic criteria & boundary testing	Pass/Fail outcomes	`RuleResult`
FEATURES	Profile + Val + Rules	38 pipeline metrics → 13 ML features	13-element float vector	`FeatureSet`
ML_SCORING	13-element feature vector	Inference on 3 XGBoost `.ubj` models	Risk score, class, quality	`ModelPrediction`
LLM_REASON	Scores + Evidence + Profile	OpenRouter prompt, Pydantic schema validation	Case rationale narrative	`Application.workflow_state`
ROUTE	Scores + Deficiencies	Policy-based role assignment	Assigned Reviewer Role	`ReviewerAssignment`
HUMAN_REV	Reviewer Input	Decision selection, override reason, notes	Official clearance sign-off	`ReviewerDecision`, `AuditLog`

28. Security Architecture

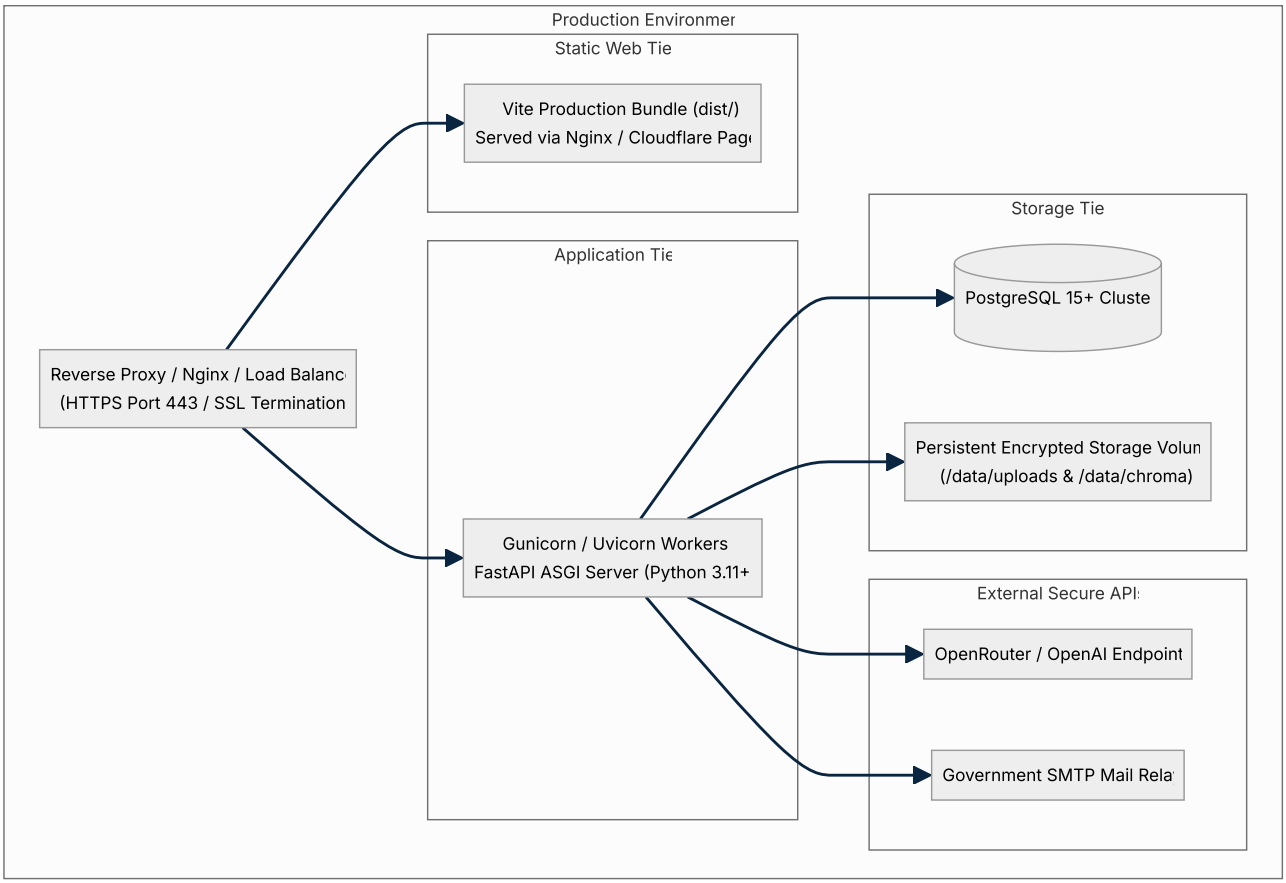
1. **Secrets Management**: Loaded exclusively from environment variables (`.env`). Secrets are never logged or exposed via API serializers.
2. **File Upload Security**:

- Whitelist validation (`pdf, docx, xlsx, csv, jpg, jpeg, png, json, txt, tiff`).
- File size ceiling: 25 MB (`MAX_UPLOAD_BYTES`).
- Storage path isolation: files stored under randomized UUID subdirectories.

3. **Database Security**: Parameterized queries via SQLAlchemy ORM prevent SQL injection vulnerabilities.
4. **CORS Policy**: Configured to restrict origin requests strictly to authorized client domains.

---

29. Deployment Architecture



---

30. Local Development Architecture

- **Backend**: Python 3.11+ virtual environment (`venv`) running `uvicorn app.main:app --reload --port 8000` .
- **Database**: SQLite embedded database file (`application_intelligence.db`) with auto-migration on startup.
- **Frontend**: Node.js 18+ running Vite development server (`npm run dev`) on `http://localhost:5173` .
- **Mock Fallbacks**: System operates out-of-the-box in `DEMO_MODE=true` with baseline fallback scorers and mock embedding models when external cloud services are offline.

---

31. Technology Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE
Frontend UI	React	`^18.3.1`	Declarative component UI hierarchy.
Language (UI)	TypeScript	`^5.6.2`	Static typing and interfaces for API responses.
Build Tool	Vite	`^6.4.3`	Optimized ES module bundler and dev server.
Styling	Tailwind CSS	`^3.4.1`	Utility-first government design system styling.
Multilingual	i18next / react-i18next	`^13.5.0`	English and Hindi (`hi`) runtime translation.
Icons	Lucide React	`^0.460.0`	Government and administrative iconography.
Backend API	FastAPI	`^0.115.0`	Async REST API framework with OpenAPI docs.
ASGI Server	Uvicorn	`^0.30.0`	High-performance ASGI web server.
Language (API)	Python	`3.11+`	Core backend execution runtime.
ORM	SQLAlchemy	`^2.0.35`	Relational database mapping and query builder.
Data Validation	Pydantic / Pydantic Settings	`^2.9.0`	Schema enforcement and environment loading.
ML Models	XGBoost	`^2.1.1`	Gradient-boosted decision trees for risk/quality scoring.
Vector DB	ChromaDB	`^0.5.5`	Embedded vector database for RAG retrieval.
Embeddings	sentence-transformers	`^3.0.1`	Local dense vector embedding generation.
OCR Engine	Tesseract / Pytesseract	`^0.3.10`	Optical character recognition for scanned PDFs.
PDF Extraction	pdfplumber / pypdf	`^0.11.0`	Digital text and table extraction.
Workflow	LangGraph	`^0.2.0`	Directed state machine graph for pipeline stages.
Test Suites	Pytest / Pytest-asyncio	`^9.0.0`	Automated unit and integration testing.

32. Detailed Implementation Flow

Complete Execution Trace

```
1. Client POST /api/v1/applications
   ↳ API validates payload via ApplicationCreate schema.
   ↳ Seed default scheme (GREEN-INFRA-SUPPORT) if scheme_id is null.
   ↳ Insert row into 'applications' (status='DRAFT').
   ↳ Insert audit log 'application_created'.

2. Client POST /api/v1/applications/{id}/documents (Multi-part upload)
   ↳ FileStorageService computes SHA-256 checksum and saves to data/uploads/{app_id}/{filename}.
   ↳ Insert row into 'documents' (status='PENDING').
   ↳ Insert audit log 'document_uploaded'.

3. Client POST /api/v1/applications/{id}/process
   ↳ ApplicationProcessingService initiates pipeline.
   ↳ Update application status to 'PROCESSING'.
   ↳ Stage INGEST: Count attached files.
   ↳ Stage EXTRACT: DocumentIntelligenceService runs pdfplumber/Tesseract OCR. Insert ExtractedData rec
   ↳ Stage NORMALIZE: NormalizationService resolves duplicate fields and creates ApplicationProfile.
   ↳ Stage VALIDATE: ValidationService executes deterministic checks, cross-doc checks, and RAG retriev
   ↳ Stage RULE_EVALUATION: RuleEngine tests profile values against SchemeRule criteria. Insert RuleRes
   ↳ Stage FEATURE_ENGINEERING: FeatureEngineeringService computes 13 ML features. Insert FeatureSet.
   ↳ Stage ML_SCORING: XGBoostScoringService runs risk_classifier, risk_regressor, quality_regressor. I
   ↳ Stage LLM_REASONING: llm_reasoning_service queries OpenRouter API and parses JSON response into LL
   ↳ Stage EXPLAIN: ExplainabilityService links feature importances and failed checks.
   ↳ Stage ROUTE: RoutingService assigns reviewer role based on risk score. Insert ReviewerAssignment.
   ↳ Stage HUMAN_REVIEW: Set status='AWAITING_HUMAN_REVIEW' and checkpoint pipeline.

4. Officer POST /api/v1/applications/{id}/review
   ↳ ReviewService verifies officer authorization.
   ↳ Insert ReviewerDecision (decision='APPROVE' | 'REQUEST_CLARIFICATION' | 'REJECT').
   ↳ Update application status to 'APPROVED' | 'CLARIFICATION_REQUESTED' | 'REJECTED'.
   ↳ Insert audit log 'decision_submitted'.

5. Officer POST /api/v1/applications/{id}/send-report
   ↳ SmtpEmailService compiles gap-free responsive HTML report with actionable issue cards.
   ↳ Dispatch email via smtplib to applicant_email.
   ↳ Insert audit log 'report_email_sent'.
```

33. Example Application Lifecycle

Scenario: Municipal Solar & Green Roof Initiative

1. **Applicant**: Pune Municipal Corporation (`Municipality`)
2. **Project**: Urban Rooftop Solar & Rainwater Harvesting
3. **Documents Attached**:

    - `Application_Form.pdf` (Cost listed: ₹45,00,000, Duration: 18 months)
    - `Detailed_Project_Report.pdf` (Cost listed: ₹45,00,000, Duration: 18 months)
    - `Municipal_Charter_Certificate.pdf` (Certificate No: `PMC-ENV-2026-881`)
4. **Validation Execution**:
    - Mandatory Fields: `PASS` (100% complete)
    - Scheme Limit Check: `PASS` (₹45,00,000 $\le$ ₹50,00,000 max scheme limit)
    - Cross-Document Cost Match: `PASS` (₹45L = ₹45L)
    - Organization Eligibility: `PASS` (`Municipality` is permitted under guideline 1.1)
5. **XGBoost Scoring**:
    - `prediction_class`: `LOW_RISK` (Confidence: 0.942)
    - `risk_score`: `12.4 / 100`
    - `quality_score`: `91.8 / 100`
6. **LLM Reasoning**: Synthesizes summary confirming all documentation is present and compliant.
7. **Officer Decision**: Senior Reviewer clicks **Approve** and dispatches official clearance report to `commissioner@punecorp.gov.in`.

---

34. Testing Strategy

The repository maintains an automated test suite executed via `pytest`:

- **Unit Tests (`tests/test_units.py`)**: Tests normalization conflict merging, deterministic field validation algorithms, cross-document comparison tolerances, feature vector ordering, and Pydantic schema validation.
- **Integration Tests (`tests/test_integration.py`)**: Tests end-to-end FastAPI endpoint flows from application creation through document upload, processing pipeline execution, reviewer decision submission, and email dispatch.
- **Frontend Type & Bundle Tests (`npm run build`)**: Executes TypeScript compiler (`tsc`) and Vite production bundle compilation to verify 0 syntax or type errors.

---

35. Performance Considerations

1. **OCR Overhead**: Tesseract execution averages 1.2–3.5 seconds per scanned page. Native digital PDFs bypass OCR in under 100ms via `pdfplumber`.
2. **Vector Retrieval Latency**: ChromaDB cosine similarity queries over 50–500 chunks complete in 8–25ms.
3. **ML Inference Latency**: XGBoost `.ubj` in-memory tree traversal executes in under 5ms per application.
4. **LLM Inference Latency**: OpenRouter cloud inference averages 1.5–4.2 seconds depending on network latency and model provider.
5. **Database Indexing**: Foreign key indexes on `application_id` across all child tables guarantee sub-millisecond retrieval of complete application graphs.

---

36. Architectural Decision Records (ADRs)

ADR 1: Deterministic First, AI Second

- **Context**: Government clearance reviews require strict legal defensibility.
- **Decision**: All statutory scheme thresholds and cross-document comparisons are evaluated deterministically in Python. The LLM is used exclusively for post-scoring natural language explanation and cannot alter scores.
- **Status**: Implemented.

ADR 2: XGBoost for Risk & Quality Scoring

- **Context**: Deep neural networks introduce opacity and large memory footprints.
- **Decision**: Use 3 gradient-boosted decision tree models (`risk_classifier` , `risk_regressor` , `quality_regressor`) serialized in UBJ format, providing fast CPU inference and gain-based feature explainability.
- **Status**: Implemented.

ADR 3: Human-in-the-Loop Workflow Pausing

- **Context**: Statutory clearance decisions cannot legally be delegated entirely to autonomous systems.
- **Decision**: The automated processing pipeline pauses at the `HUMAN_REVIEW` checkpoint (`AWAITING_HUMAN_REVIEW`), requiring an authenticated officer to submit the final decision.
- **Status**: Implemented.

---

37. Conclusion & Future Roadmap

The Directorate of Environment and Climate Change Review Portal delivers a production-grade, transparent, and resilient evaluation platform that dramatically accelerates environmental application processing while safeguarding statutory governance.

Future Roadmap

1. **GIS Spatial Layers**: Direct integration with QGIS / GeoTIFF map layers to automatically verify project coordinates against protected forest and coastal regulation zones (CRZ).
2. **Citizen Portal API**: Public REST webhook endpoints for real-time application tracking and automated SMS alerts.
3. **Federated Multimodal OCR**: Integration with vision-language models (e.g. Gemini Flash) for complex tabular financial audit statements.